

Lista de Exercícios — Aula 06

Lista Dinâmica Encadeada

Instruções: As questões a seguir pedem a implementação de novas funções sobre a lista dinâmica encadeada de tarefas apresentada abaixo. Implemente cada função em C padrão, compile com gcc -Wall -Wextra -std=c11.

A struct elemento (com os campos dados e prox) está definida no arquivo .c da biblioteca e não é visível aqui: assim como em aula, o tipo ListaTarefas é opaco, e as novas funções só podem acessar os nós através dos ponteiros prox.

Biblioteca base (já implementada)

```
struct tarefa {
    int  codigo;
    char descricao[40];
    int  prioridade; /* 1 = mais urgente ... 5 = menos urgente */
};

typedef struct elemento* ListaTarefas;

ListaTarefas* cria_lista(void);
void libera_lista(ListaTarefas* li);
int tamanho_lista(ListaTarefas* li);
int lista_vazia(ListaTarefas* li);
int lista_cheia(ListaTarefas* li);
int insere_tarefa_inicio(ListaTarefas* li, struct tarefa t);
int insere_tarefa_final(ListaTarefas* li, struct tarefa t);
int insere_tarefa_ordenada(ListaTarefas* li, struct tarefa t);
int remove_tarefa_inicio(ListaTarefas* li);
int remove_tarefa_final(ListaTarefas* li);
int remove_tarefa(ListaTarefas* li, int codigo);
int busca_tarefa_pos(ListaTarefas* li, int pos, struct tarefa *t);
int busca_tarefa_cod(ListaTarefas* li, int codigo, struct tarefa *t);
```

tamanho_lista, lista_vazia e lista_cheia devolvem -1 quando li é NULL; insere_tarefa_ordenada ordena a lista pelo campo prioridade, do menor para o maior.

Protótipos a implementar

```
int conta_tarefas_prioridade(ListaTarefas* li, int prioridade);
int tarefa_mais_urgente(ListaTarefas* li, struct tarefa *t);
int busca_tarefa_desc(ListaTarefas* li, char *texto, struct tarefa *t);
int insere_tarefa_final_prioridade(ListaTarefas* li, struct tarefa t);
int remove_tarefas_prioridade(ListaTarefas* li, int prioridade);
int inverte_lista(ListaTarefas* li);
int remove_tarefa_pos(ListaTarefas* li, int pos);
int mescla_tarefas(ListaTarefas* dst, ListaTarefas* src);
```

Questão 1

```
int conta_tarefas_prioridade(ListaTarefas* li, int prioridade);
```

Devolve quantas tarefas da lista têm exatamente a prioridade informada. Devolve -1 se li for NULL.

Questão 2

```
int tarefa_mais_urgente(ListaTarefas* li, struct tarefa *t);
```

Copia para *t a tarefa de menor valor de prioridade (a mais urgente). Em caso de empate, copia a primeira encontrada no percurso. Devolve 1 em caso de sucesso e 0 se a lista for inválida ou estiver vazia.

Questão 3

```
int busca_tarefa_desc(ListaTarefas* li, char *texto, struct tarefa *t);
```

Percorre a lista procurando a primeira tarefa cuja descrição contenha texto como substring, e a copia para *t. Devolve 1 em caso de sucesso e 0 caso contrário.

Dica: Você pode usar a função `strstr` da biblioteca `<string.h>` para testar se uma string contém outra.

Questão 4

```
int insere_tarefa_final_prioridade(ListaTarefas* li, struct tarefa t);
```

Insere a tarefa t imediatamente após a última tarefa da lista que tiver a mesma prioridade de t, de modo a manter agrupadas as tarefas de mesma prioridade. Se não houver nenhuma tarefa com essa prioridade na lista, insere t no final (mesmo comportamento de `insere_tarefa_final`). Devolve 1 em caso de sucesso e 0 em caso de falha.

Dica: É preciso percorrer a lista inteira: a última ocorrência de uma prioridade pode estar em qualquer posição, não apenas próxima do início.

Questão 5

```
int remove_tarefas_prioridade(ListaTarefas* li, int prioridade);
```

Remove da lista todas as tarefas que tiverem a prioridade informada, não apenas a primeira, liberando a memória de cada uma. Devolve quantas tarefas foram removidas, ou -1 se li for NULL.

Dica: A lista pode ter duas ou mais tarefas seguidas com a prioridade removida, inclusive no início ou no final. Cuidado ao decidir quando avançar ant e quando manter ant parado.

Questão 6

```
int inverte_lista(ListaTarefas* li);
```

Inverte a ordem dos elementos da lista, alterando apenas os ponteiros prox dos nós já existentes. Não é permitido alocar nenhum nó novo, nem copiar os dados de uma tarefa para outro nó. Devolve 1 em caso de sucesso e 0 se li for NULL.

Dica: Uma lista vazia ou com um único elemento já está, trivialmente, invertida.

Questão 7

```
int remove_tarefa_pos(ListaTarefas* li, int pos);
```

Remove a tarefa que está na posição pos da lista, contada a partir de 1, sem depender do código da tarefa. Devolve 1 em caso de sucesso e 0 se a lista for inválida, vazia, ou se pos não corresponder a nenhuma posição existente.

Questão 8

```
int mescla_tarefas(ListaTarefas* dst, ListaTarefas* src);
```

Transfere todas as tarefas de src para o final de dst, sem copiar dados nem alocar nenhum nó novo: os próprios nós de src passam a fazer parte de dst. Ao final, src deve ficar vazia (mas continuar sendo uma lista válida, e não NULL). Devolve a quantidade de tarefas transferidas, ou -1 se dst ou src forem NULL.

Dica: Diferente da lista sequencial estática, aqui não é preciso copiar elemento por elemento: basta religar um ponteiro.